

REFYNE AI — Enterprise Requirement Engineering & Document Intelligence Platform

Enterprise AI workflow and document intelligence platform for requirement discovery, analysis, validation, governance and formal document generation.

Source: REEYABORIKAR/Assistant GitHub repository, using the repository README, product/architecture specifications and implemented frontend/backend foundation.

Table of Contents

- 1. Overview.....3
- 2. Implemented Foundation vs. Full Specification3
- 3. Architecture4
- 4. Technology Stack.....5
- 5. Project Structure.....5
- 6. End-to-End Workflow5
- 7. Authentication & Tenant Workspace.....6
- 8. Conversational AI Workspace.....6
- 9. Document Intelligence & Audit Dashboard6
- 10. Requirement Repository6
- 11. Risk, Security & Compliance Analysis7
- 12. AI Agent Architecture.....7
- 13. Prompt Injection & Tool Security.....7
- 14. Integrations8
- 15. Database Architecture.....8
- 16. Document Generation8
- 17. Async Processing & Realtime Updates9
- 18. API Surface.....9
- 19. Security Controls.....9
- 20. Known Gaps & Decisions10
- 21. Setup & Run10
- 22. Testing10
- 23. Quick Viva / Demo Answers.....10
- 24. Conclusion.....11

1. Overview

1.1 Problem

Software requirements are often distributed across documents, conversations, tickets and external knowledge systems. This makes requirement discovery, validation, traceability, risk analysis and formal document preparation difficult to manage consistently.

1.2 Solution

REFYNE AI provides a chat-first workspace that accepts requirement inputs and documents, analyzes them through controlled workflow stages and specialized AI agents, validates outputs, supports human review and approval, and generates formal engineering documents.

1.3 Core user journey

Register → Login → AI Workspace → Create Project → Upload Requirements → Chat With AI → Requirement Analysis → Supervisor → Conditional Workflow → Security → Compliance → Risk → Data Availability → Validation → Human Review → Approval → Document Generation → Document Validation → PDF → Version → Audit

1.4 Primary capabilities

Capability	Repository-defined scope
Chat	Natural-language interaction, streaming, conversation history, attachments, contextual actions, document chat and project context
Workflow	Configurable capabilities, conditional/stateful workflow, retries, auditability and human-in-the-loop
AI agents	Supervisor, Requirement, Security, Compliance, Risk, Data Availability, Validation and Document Supervisor
Integrations	Jira, Confluence and Notion
Documents	BRD, SRS, FRD, NFRD, RTM, Test Plan, Test Cases, Risk, Security, Compliance and Technical Documentation
Enterprise	Multi-tenancy, RBAC/ABAC, audit, approvals, observability, security and versioning

2. Implemented Foundation vs. Full Specification

The repository contains a documented target architecture as well as an implemented foundation. The distinction matters when describing the project in a demo or viva.

Area	Repository evidence
Backend	FastAPI application boundary is present; repository documentation defines a larger service architecture
Frontend	React 18 SPA with React Router, Axios, authentication context and tenant context
Authentication	Login/register UI and JWT token handling are implemented in the frontend; the broader security contract is specified
Tenant isolation	Tenant isolation middleware is implemented and tested according to FINAL_SUMMARY.md
Workflow	19-state authoritative workflow model is specified
Agents	8-agent architecture is specified, with structured outputs and validation contracts
Document generation	11 document types are specified, with PDF, DOCX and XLSX rendering services
Integrations	Jira, Confluence and Notion tool contracts are specified; tool registry currently fails closed by default
Reports	Reporting service is explicitly marked BLOCKED in the architecture specification

3. Architecture

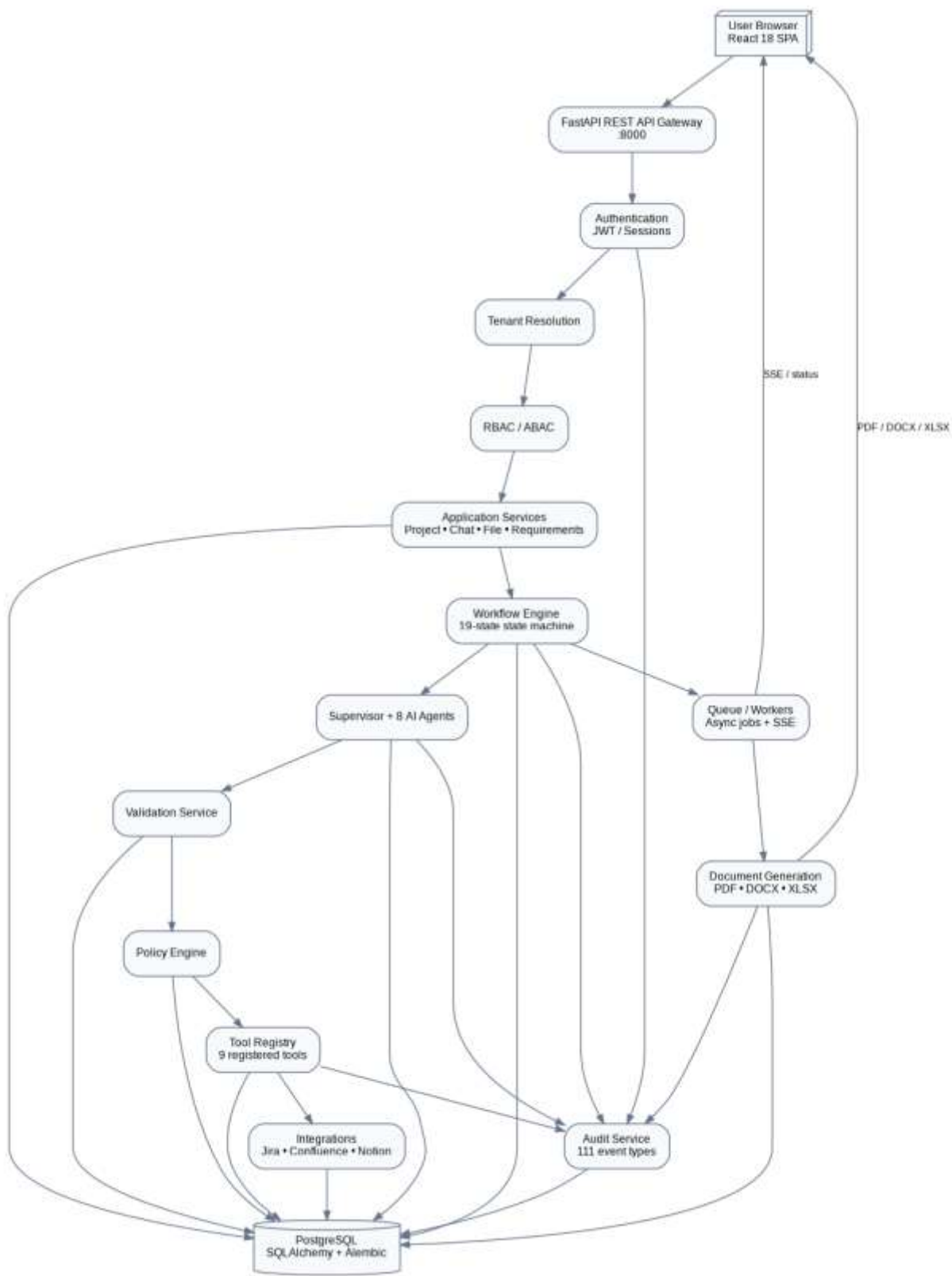


Figure 1. REFYNE AI layered system architecture

The documented request pipeline is ordered as: Frontend → API Gateway → Authentication → Tenant Resolution → RBAC/ABAC → Application Services → Workflow Engine → Supervisor → Agents → Validation → Policy Engine → Tool Registry →

Integrations. Tenant scope is derived from the authenticated session, and external operations are required to pass through the Tool Registry.

4. Technology Stack

Layer	Technology / role
Frontend	React 18, React Router, Tailwind CSS, Lucide Icons, Axios
Backend API	Python 3.10+, FastAPI, Pydantic
Persistence	PostgreSQL, SQLAlchemy ORM, Alembic
AI intelligence	Groq Cloud API, including documented model configurations such as groq/compound-mini and openai/gpt-oss-120b
Document rendering	ReportLab for PDF; separate DOCX and XLSX services
Async processing	Queue + workers, with SSE for realtime server-to-client progress/events
Containerization	Docker and Docker Compose

5. Project Structure

```
Assistant/  
├── backend/  
│   ├── src/refyne/  
│   ├── tests/  
│   ├── data/  
│   │   ├── audits/  
│   │   └── extracted/  
│   ├── Dockerfile  
│   └── alembic.ini  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── contexts/  
│   │   ├── pages/  
│   │   └── services/  
│   ├── package.json  
│   └── Dockerfile  
├── docs/  
│   ├── product / UI / backend / workflow / DB / API / agent specifications  
│   └── ui/ screenshots and design references  
└── workers/
```

6. End-to-End Workflow



Figure 2. Authoritative REFYNE AI workflow state path

The workflow specification defines 19 states. Optional states include Security Analysis, Compliance Analysis and Risk Analysis. Human Review and Approval are conditional. Terminal states are Completed, Failed, Rejected and Cancelled. RETRY is a transition trigger rather than a state.

7. Authentication & Tenant Workspace

7.1 Authentication

The frontend exposes /login and /register. AuthContext loads the stored JWT token, calls /api/v1/auth/me when a token exists, stores access_token in localStorage after login, and clears the token on logout.

7.2 Tenant awareness

TenantContext calls /api/v1/tenants/current after authentication and exposes the current tenant to the UI. The backend security specification requires tenant resolution from the authenticated session rather than trusting a client-provided tenant identifier.

8. Conversational AI Workspace

The documented /chat experience is the primary interaction surface. It includes chat history, attachments, contextual generation actions, uploaded files, generated documents and quick AI actions. The system is designed for natural-language requests such as analyzing requirements, identifying security/compliance gaps, generating user stories and producing BRD/SRS/RTM artefacts.

8.1 File ingestion

The specification defines a file pipeline covering upload, validation/scanning, quarantine, extraction, persistence and downstream use. Supported document inputs described for the chat experience include PDF, DOCX, TXT, JSON, CSV and Markdown.

8.2 Document intelligence

Extracted document content becomes authorized document data or untrusted external content depending on its provenance. The agent specification explicitly treats document text as data, not system instructions, to reduce prompt-injection risk.

9. Document Intelligence & Audit Dashboard

The dashboard is designed to expose document readiness, clarity/precision, completeness, security/SAIF, traceability, risks, ambiguities, missing sections and RTM mappings. Audit output is intended to be tied to stored project/document records and persisted for later inspection.

Audit area	Purpose
Readiness	Overall document preparation state
Clarity & Precision	Ambiguity and terminology quality
Completeness	Coverage of functional and non-functional scope
Security & SAIF	Security controls, authentication, privacy and related findings
RTM Traceability	Requirement-to-design/test/risk/evidence mapping
Risk factors	Security, architectural and data-privacy failure modes
Conflicts & ambiguities	Conflicting or vague requirements
Missing sections & gaps	Missing NFRs, SLAs, DR or other specified sections

10. Requirement Repository

The Requirements area is specified as a repository for structured requirements, including type, status, priority and traceability relationships. Requirement identity is assigned by the platform rather than by an agent.

Requirement type examples	Repository enum includes
Business	BUSINESS
Functional	FUNCTIONAL
Non-functional	NON_FUNCTIONAL
Security	SECURITY
Compliance	COMPLIANCE

Data	DATA
Interface	INTERFACE

11. Risk, Security & Compliance Analysis

The platform separates risk, security and compliance analysis into dedicated agent responsibilities. Risk scoring must be driven by configured policy. The repository explicitly leaves risk score nullable when no configured scoring policy exists rather than inventing a scoring rule.

11.1 Security analysis

Security findings include severity, evidence, affected requirements, controls, remediation and status. Evidence-backed assertions are required by the documented validation and database contracts.

11.2 Compliance analysis

Compliance findings use a compliance status model and evidence-backed validation. The framework catalogue remains a documented missing contract in the current specification set.

11.3 Risk analysis

Risk analysis records likelihood and severity along with mitigation, ownership and lifecycle status. A derived numeric risk score is only valid when the corresponding policy is configured.

12. AI Agent Architecture

Agent	Purpose / documented role
Supervisor	Controls workflow decisions and coordination; no direct tools in the documented contract
Requirement	Extracts and structures software requirements
Security	Analyzes security findings and controls
Compliance	Analyzes compliance requirements and evidence
Risk	Identifies and manages project risks
Data Availability	Determines whether required data is available, partial or missing
Validation	Validates agent outputs, documents, traceability and policy compliance
Document Supervisor	Coordinates document type, sources, required sections and generation context

Each agent is documented with structured input/output, validation, retry/failure behavior, audit requirements and escalation rules. Agent runs record provider, model, configuration, timestamps, output and validation information.

13. Prompt Injection & Tool Security

The security specification establishes explicit context buckets. Only SYSTEM_INSTRUCTIONS may instruct the agent. User input, project data, authorized document data, integration data, untrusted external content and tool output are treated as data.

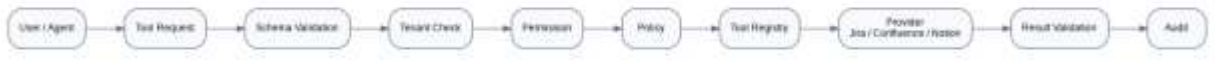


Figure 3. Controlled external-tool execution chain

Nine tools are registered across Jira, Confluence and Notion. Tool execution is gated by schema validation, tenant checks, permissions, policy and Tool Registry controls. The repository documents enabled=false and empty allowed_agents as the current fail-closed position.

14. Integrations

Integration	Capabilities documented	Current documented constraint
Jira	Search, get, create, update, link	OAuth contract exists; external writes require authorization and idempotency
Confluence	Search and retrieve pages	Authentication method is a missing contract; read-only
Notion	Search and retrieve pages	Authentication method is a missing contract; read-only

Disconnected integrations must not return fabricated results. External operation attempts are auditable.

15. Database Architecture

The documented database engine is PostgreSQL with Alembic migrations. The schema is multi-tenant aware and distinguishes ROOT, GLOBAL, DIRECT and DERIVED entities. Examples include tenants, users, projects, conversations, files, workflow definitions/runs, requirements, risks, security findings, compliance findings, documents, reviews, approvals, integrations, audit events and jobs.

15.1 Key database groups

Group	Representative entities
Identity & tenancy	tenants, users, tenant_members, roles, user_roles, sessions
Project & collaboration	projects, project_members, conversations, messages
Files & extraction	files, file_chunks
Workflow	workflow_definitions, workflow_steps, workflow_runs, workflow_state_transitions, workflow_events
AI execution	agent_runs, validation_runs, model_configurations
Requirements & analysis	requirements, requirement_links, risks, security_findings, compliance_findings, evidence
Documents	documents, document_versions, document_sections, document_sources, generation_runs, document_renditions
Review & approval	reviews, review_assignments, review_sections, review_comments, approvals, approval_steps
Integrations & tools	integrations, integration_credentials, tool_registry, tool_bindings, tool_calls
Governance	policies, policy_versions, policy_rules, audit_events, quotas, jobs

16. Document Generation

Document type	Documented availability
BRD	Required
SRS	Required
FRD	Required
NFRD	Required
RTM	Required
TEST_PLAN	Required
TEST_CASES	Required
RISK_ASSESSMENT	Required
SECURITY_ASSESSMENT	Required
COMPLIANCE_ASSESSMENT	Required

The architecture defines separate rendering services for PDF, DOCX and XLSX. XLSX generation is required for RTM and TEST_CASES. Stored renditions include file size and SHA-256 checksum, and approved document versions are immutable.

17. Async Processing & Realtime Updates

Long-running file extraction, workflow execution and document generation are designed as asynchronous jobs. The documented sequence is API → create job → queue → worker → database → event → SSE → frontend. Dead-lettered jobs are terminal failures, and streams are re-authorized and tenant-scoped.

18. API Surface

The repository's API specification covers authentication, tenant/current context, projects, chat/conversations, files, workflows/workflow-runs, requirements, analysis, test artefacts, traceability, documents/generation, reviews, approvals, integrations, tools, admin, dashboard and reporting.

Area	Representative endpoints / operations
Authentication	POST /api/v1/auth/register, POST /api/v1/auth/login, GET /api/v1/auth/me
Tenant	GET /api/v1/tenants/current
Files	POST /files, file status / extraction operations
Workflows	Create definitions, start runs, inspect runs, retry/cancel, stream events
Documents	Generate, validate, review, approve/reject, revisions and renditions
Integrations	Jira connect/callback and provider operations through the tool layer
Governance	Admin users/roles/workflows/tools/models/templates/policies/quota and audit access

19. Security Controls

Control	Repository-defined approach
Default deny	Permission not explicitly granted is denied
Authentication	JWT/session based authentication contract
Tenant isolation	Authenticated-session-derived tenant scope; strategy decision remains documented
Password storage	Passwords are hashed and never stored plaintext
File security	Scan/quarantine pipeline; uploaded files must never be executed as code
Secret management	Credentials referenced through secret-management contracts; raw secrets not returned
Rate limiting	Specified as required, but exact policy remains an outstanding decision
LLM output constraints	Structured schemas and validation before downstream use
Prompt injection	Context bucket trust model and explicit injection tests
Tool security	Registry, schema, permission, policy, enabled and allowed-agent gates
Audit	Append-only audit records and a large event catalogue

20. Known Gaps & Decisions

The repository itself records unresolved decisions and missing contracts. These should not be presented as completed capabilities.

Item	Repository status
Tenant isolation mechanism	Decision required: application filtering vs PostgreSQL RLS vs schema-per-tenant
Role model	Decision required: role set, assignment model and permissions
Retry policy	Decision required for automatic retry limits/backoff/exhaustion behavior
Risk scoring	Decision required; derived score remains unavailable without a policy
Confluence / Notion authentication	Missing contract
Permission catalogue	Additional endpoint permissions remain a documented missing contract
Reporting	Explicitly BLOCKED in the architecture specification
Notification service	Table exists in the schema, but no service/endpoint contract is defined

21. Setup & Run

21.1 Backend

```
cd backend
pip install -r requirements.txt
uvicorn refyne.main:app --reload
```

21.2 Frontend

```
cd frontend
npm install
npm start
```

The repository documentation identifies Python 3.10+ and a frontend development server on port 3000 for the implemented React foundation. Environment configuration should be supplied locally and secrets must not be committed.

22. Testing

The repository includes backend tests and a dedicated tenant-isolation test suite. The documented testing scope covers tenant isolation, IDOR, privilege escalation, authorization bypass, session security, upload abuse, prompt injection, tool authorization, integration scope, secret exposure and rate limiting, while explicitly marking some areas as partial or blocked where contracts remain unresolved.

```
cd backend
PYTHONPATH=./src python -m pytest tests/test_tenant_isolation.py -v
```

23. Quick Viva / Demo Answers

Question	Answer grounded in the repository
What is REFYNE AI?	An enterprise AI workflow and document intelligence platform focused on requirement engineering and formal specification.
Why chat-first?	The primary interaction is natural language, allowing one request to trigger controlled requirement, analysis and documentation workflows.
How is security handled?	Authentication, tenant resolution, RBAC/ABAC, policy checks, tool registry controls and audit are placed in the

	request pipeline.
How many AI agents are defined?	Eight: Supervisor, Requirement, Security, Compliance, Risk, Data Availability, Validation and Document Supervisor.
How many workflow states are defined?	Nineteen, including terminal Rejected in the revised state machine.
What documents can be generated?	Eleven defined document types, with PDF for documents and XLSX specifically for RTM and Test Cases.
How are external tools controlled?	Through the Tool Registry with schema validation, tenant, permission, policy and execution controls.
How is prompt injection handled?	Document and external text are treated as data, not instructions, and external effects must pass the independent authorization chain.
Where is the data stored?	The documented persistence layer is PostgreSQL, with file/extracted content and audit artefacts also represented in backend storage.
What is currently blocked?	Several contracts remain unresolved, including reporting and certain integration/authentication/permission decisions.

24. Conclusion

REFYNE AI is structured as a controlled enterprise workflow platform rather than a single AI prompt. Its repository combines a React workspace, FastAPI backend boundary, PostgreSQL persistence model, stateful workflow engine, specialized AI agents, validation and policy layers, controlled third-party integrations, document rendering and audit governance. The documentation also explicitly records implementation boundaries and unresolved decisions so the system can be extended without silently inventing behavior.